DFS.

Wednesday, 8 May 2024 3:56 PM

(informal vanille version)

```
DFS ( G, u).
    For each v ∈ Adj (u)
        if (v is not visited)
            DFS (v).
```

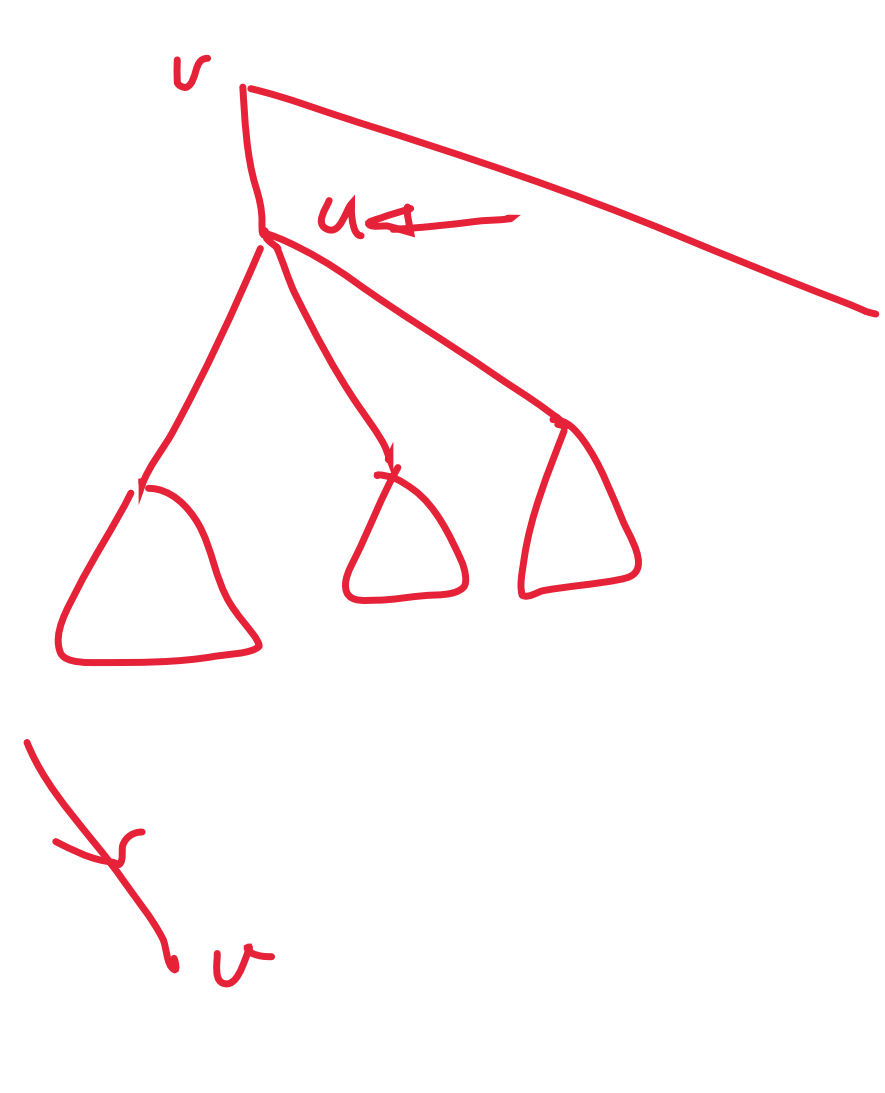
Exploration of a graph.
Systematic visit (enumeration) of vertices and edges.
Graph is a complex structure. We need to process them systematically and incrementally ...

visited to unvisited – Tree edge.
visited to visited – non Tree edge.

Three states – Col – White
Gray
Black. }

Tree edges – (p(v), v).

$u \xrightarrow{\quad} v$ parent of v.
u = p(v).



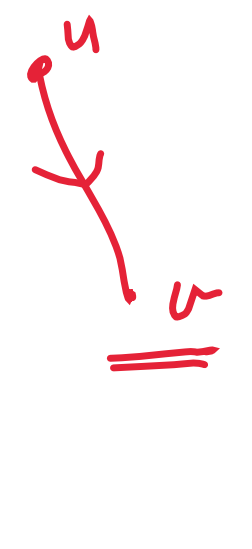
vertex v. v.d – discovery time
v.f – finish time.

u – parent of v.
v.p — parent of v.

DFS (undirected graph).

```
DFS (G).
    For each u ∈ V
        u.color = White.
        u.p     = NIL;
    Time = 0;
    For each u ∈ V
        if ( u.color == White)
            DFS (G,u).

DFS (G,u)
    Time = Time +1 ;
    u.d = Time ;
    u.color = Gray ;
    For each v ∈ Adj (u)
        if ( v.color == White)
            v.p = u
            DFS (G, v)
    Time = Time +1 ;
    u.f = Time ;
    u.Color = Black ;
```



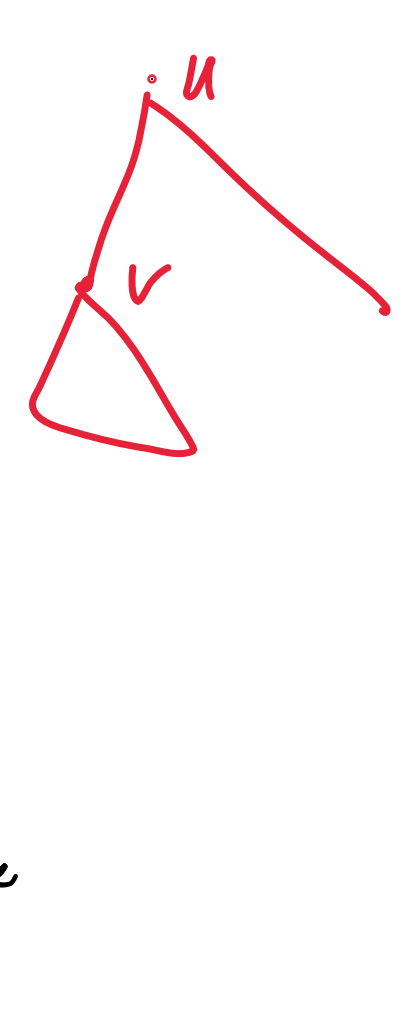
[The above one is called DFS skeleton].

[We may add more computational steps at various 'control points' of the above code and generate more information about the graph].

Extended view of computations on a graph with DFS.

```
DFS ( G, u )
    Time = Time +1
    u.d = Time
    \\ Discovery Time.
    \\ First time u is visited.
    \\ ① Process at u when u is encountered
    \\  for the first time
    \\  ( like Preorder).
    u.color = Gray     \\ u is active now.
    For each v ∈ Adj (u)
        \\ ② Include the computation to
        \\  be done for every edge here.
        \\  The current edge is ( u, v ).
        if ( v.color == White)
            \\ ③ Include the computations
            \\ to be done on tree edges
            \\ here. The Current
            \\ Tree edge is (u,v).
            v.p = u ;
            DFS (G, v) ;
            \\ ④ Include the computations
            \\ to be done while backing
            \\ from v to u
        else if ( v == u.p)
            \\ (u,v) is the second
            \\ copy of the tree edge
            \\ (v, u)
        else \\ (u,v) is a back edge
            \\ ⑤ include here the
            \\ computations to be done
            \\ for a back edge.
    Time = Time +1;
    u.f = Time
    \\ ⑥ Include the computations to be done
    \\ while leaving u. ( like post order)
    u.color = Black.
end;
```

u v
p(v) v
is tree edge
(v, p(v)) is the second copy

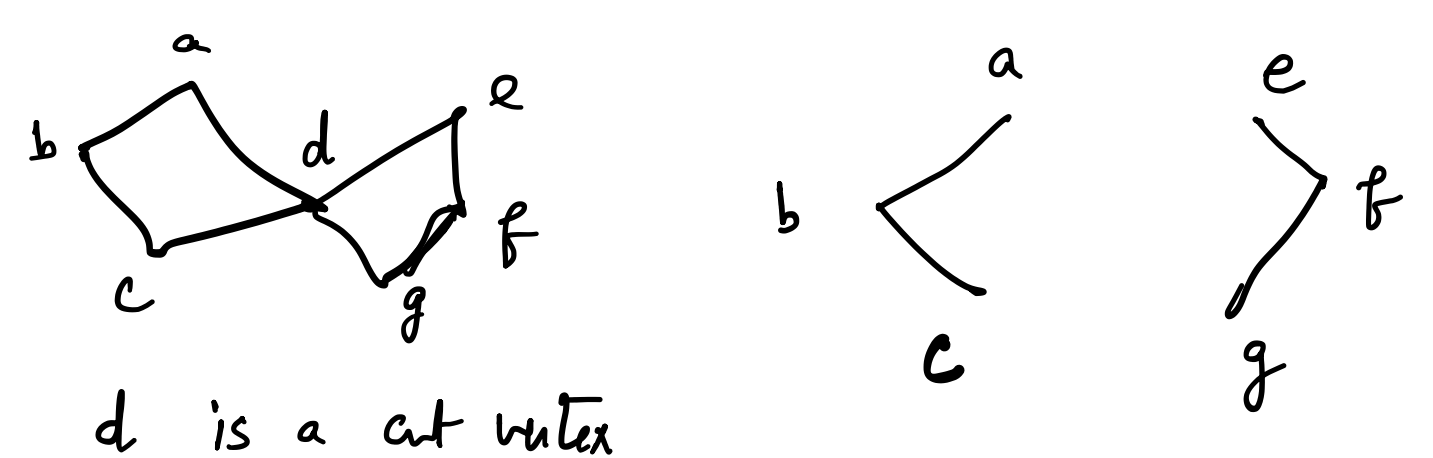


of these indicated 6 control points where additional computational steps may be added and expand the DFS code to obtain more information about the graph "on the go".

Algorithm for cut vertices:

This is a typical and canonical example of using DFS to solve graph problems.

Let G be a connected graph. A vertex v is a cut vertex if its removal disconnects G.



d is a cut vertex of G

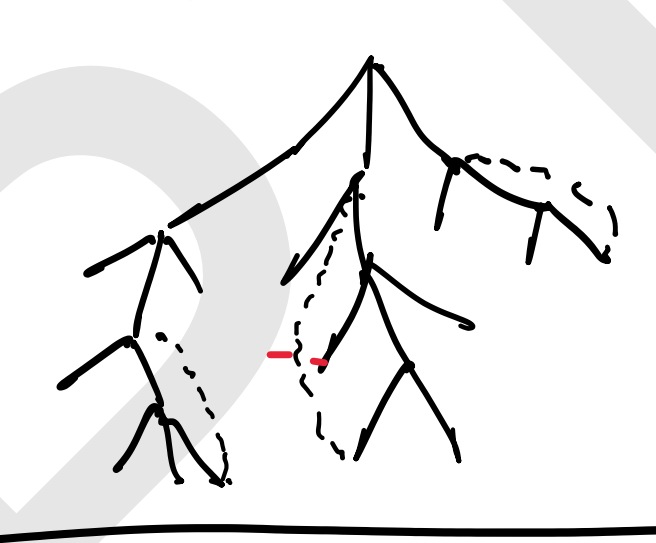
G – {d}.

A graph theoretic Charecterization (Mathematical Thinking).

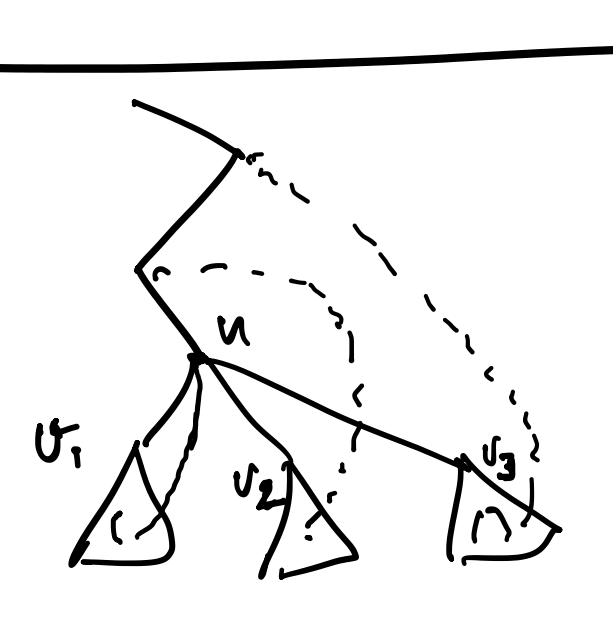
A vertex v is a cut vertex of a graph G iff ∃ a pair of vertices x, y ∋ every path between x and y contains v.

(Computationally infeasible.) .

Let us view G in terms of tree edges and non tree edges with respect to some DFS done on G.



v.cv = 0 – NOT a cut vertex.
= 1



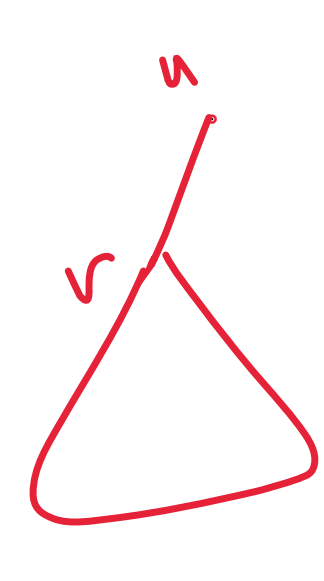
Removal of u disconnects v_1 but does not disconnect v_2 or v_3.

Low (u) = Min { v.d | v is a vertex reachable from u by using zero or more tree edges and zero or one back edge}

= Min { u.d, Min { v.d | (u,v) is a back edge },
Min { v.d | v is reachable from a descendent of u by a back edge }. }.

Observe that
Min { v.d | v is reachable from a descendant of u by a back edge }
= Min { Low (v) | v is a child of u in the DFS tree}.

Note further that all values needed to compute Low (u) are not available in one place to work on them. The values mentioned in the equation --- are generated at various times ~~and at various~~ during the execution of the program. Hence, we update Low (u) as and when a relevant value for computing Low (u) is generated.
Specifically, Low (u) may be initialised as u.d when u.d was assigned a value.



(★) Low (u) ~~Can~~ may be updated as Low (u) = Min { Low (u), v.d } where (u, v) is a back edge and this may be done when the back edge (u, v) was encountered in the DFS.

(★) Low (v) for a child v of u can be recursively computed and Low (u) is updated as Min { Low (u), Low (v) } when the control returns from v to u after finishing the DFS at the child v.

In order to carry out these steps,

we add,
u.low = u.d at control point ①

we add,
u.low = Min (u.low, v.low) at control point ④, where we are backing from v to u

and finally
we add
u.low = Min(u.low, v.d) at control point ⑤ where we know that (u, v) is a back edge.

The full details are presented in the following pseudocode.

```
DFS_LOW (G)        \\ G is connected.
    For each u ∈ V
        u.color = White;
        u.p     = NIL;
        u.d     = 0
        u.low   = 0
    Time = 0
    DFS_Low (G, u)        \\ u ∈ V is arbitrary.

DFS_Low (G, u).
    Time = Time +1 ;
① → u.d = Time ;
    u.low = u.d
    u.Color = Gray.
    For each v ∈ Adj (u)
        if ( v.color == White)
            DFS_LOW (G, v)
④ →         u.low = Min { u.low, v.low}
⑤ →     else if ( v ≠ u.p)
            u.low = Min { u.low, v.d}
    Time = Time +1 ;
    u.f = Time
    u.Color = Black .
end;
```

The low values are useful in arriving at a new charecterization for Cut vertices,

Let u be a node that not the root of the DFS Tree.
The vertex u is a cut vertex iff ∃ a child v of u in the DFS Tree such that low (v) ≥ u.d

Root of the DFS Tree is a cut vertex iff it has more than one child in the DFS Tree.

Thus, at control point ④ we do the following minor change.
u.cv = 1 if u is a cut vertex and
= 0 if u is NOT a cut vertex.
Initially, u.cv = 0 ∀ u ∈ V.

All we need to do now is add the following at control point ④.
→ if (u is not the root) and (v.low ≥ u.d)
u.cv = 1 .
(For the root node, we check if its degree is ≥ 2 in DFS tree to determine if it is a cut vertex).

Iterative Version: